机器学习与深度学习：图解速查手册

神经网络基础

从结构到一次参数更新

第一批验收样章

本批逐张核查了 12 张原始速查图，将其知识主题合并成连贯章节。图表重新绘制，补充了符号、条件、读图说明和可复算的例子。本文件仅包含这一批样章；完整工作稿仍在修订中。

原项目作者：张海飞、郭金珊、任露芹、李慧、朱靖、朱弘毅

2026 年 10 月 28 日

目录

第一章 神经网络：从结构到一次参数更新 **1**

1.1 把训练过程连起来 1

1.2 结构、前馈与隐藏层 2

1.3 神经元与前向计算 3

1.4 为什么需要非线性激活? 4

1.5 Softmax：把一组得分变为分布 5

1.6 把输出层和损失函数配对 6

1.7 反向传播：把链式法则组织起来 7

1.8 手算一次完整的训练更新 8

1.9 梯度下降与学习率 9

1.10 一页速查与自测 10

第一章　神经网络：从结构到一次参数更新

神经网络（Neural Network）把一系列可调的数学变换组合起来，将输入映射为预测。学习的关键，是利用数据调整这些变换中的参数，而不是手工规定每个中间特征。本章围绕一个问题展开：**一次预测出错后，网络怎样知道该调整哪些参数？**

> **本章阅读路线**
>
> 先看网络如何计算预测，再选择输出与损失，最后把损失的梯度传回各层并更新参数。需要的数学只有加权求和、矩阵乘法、导数和链式法则。本章用列向量表示单个样本；n 表示样本数，d 表示输入特征数，K 表示类别数。

1.1　把训练过程连起来

训练一个监督学习网络，需要训练数据、网络结构、损失函数和优化方法。损失函数回答“当前预测错了多少”，反向传播回答“参数改变会怎样影响损失”，优化器根据梯度决定“这一步怎样改参数”。

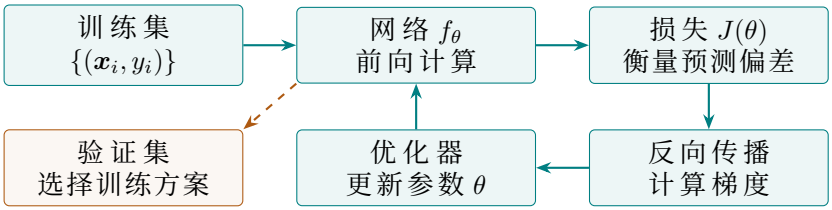


图 1.1：训练循环与验证的分工。虚线表示用验证数据评估方案，不通过该分支拟合参数。

读图：沿实线走一圈，就是一次参数更新。网络先产生预测，损失把预测与标签比较，反向传播计算梯度，优化器据此更新参数，随后才能开始下一次计算。验证集用来选择网络宽度、学习率或停止时刻；测试集留到方案确定后评估。

对于训练集 $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{n}$，一个常见目标是

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell\big(f_\theta(\boldsymbol{x}_i), y_i\big) + \lambda \Omega(\theta). \tag{1.1}$$

这里 θ 是网络全部可训练参数，f_θ 是预测模型，ℓ 是单个样本的损失，Ω 是正则化项，$\lambda \geq 0$ 控制正则化强度。本章手算例子令 $\lambda = 0$，便于看清计算过程。

> **容易混淆**
>
> 测试集不参与参数拟合、阈值选择或学习率调整。归一化、缺失值填补等需要从数据估计的步骤，也只能先在训练数据上拟合，再应用于验证集和测试集。训练损失下降，并不保证新数据上的误差下降。

1.2 结构、前馈与隐藏层

前馈神经网络（Feedforward Neural Network）的计算连接不形成有向环。全连接的多层感知机（Multilayer Perceptron, MLP）是其中一种常见形式：相邻层的每个单元与上一层的全部单元相连。卷积网络也可以是前馈网络，所以“前馈网络”和“MLP”不能在所有语境下画等号。

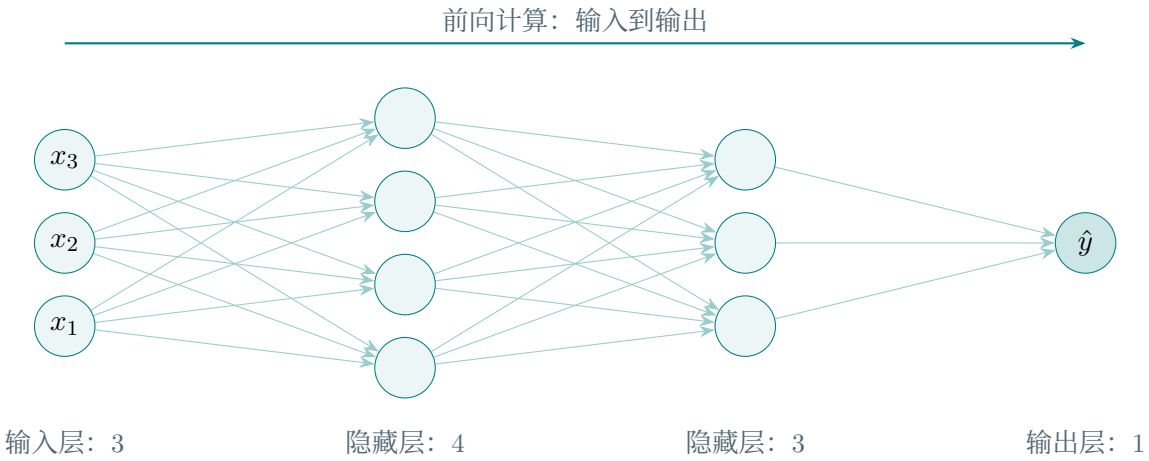


图 1.2： 一个 $3\to4\to3\to1$ 的 MLP。箭头表示计算依赖，偏置未画出。

读图： 最左侧的三个输入单元只是三个特征的入口。中间两列分别有四个和三个隐藏单元，最右侧只有一个输出。每条连接对应一个权重；每个隐藏单元和输出单元通常还有一个偏置。这些偏置不应因为图中省略而在计算时被忘掉。

隐藏层（Hidden Layer）位于输入与输出之间，其激活值是网络根据输入计算出的中间表示。“隐藏”指它们没有直接作为任务所给定的输入或目标，并不表示它们无法观察，也不表示必须另有标签。它们会在后续训练中通过最终损失的梯度间接得到调整。

宽度（Width）描述一层有多少单元；**深度（Depth）**描述计算经过多少层。本书计算参数层数时不计输入层，因此图中的网络有三个参数层、两个隐藏层。其他资料可能采用不同的计数约定，比较结构时应先确认口径。

这个网络的可训练参数总数为

$$(3\times4+4)+(4\times3+3)+(3\times1+1)=35.$$

每个括号中的前一项数连接权重，后一项数偏置。输入层没有要在这里拟合的权重或偏置。

> **结构由什么决定？**
> 输入维度由数据表示决定，输出形式由任务决定；隐藏层的层数、宽度和激活函数是建模选择。可以在验证集上比较候选结构，但不存在对所有数据都最优的网络深度。更多层或更多参数带来表达能力与计算成本的变化，不自动带来更好的泛化。

1.3 神经元与前向计算

人工神经元（Artificial Neuron）可以看成“加权求和，加偏置，再做激活”的小计算模块。权重决定输入各分量怎样组合，偏置决定变换的平移，激活函数改变组合结果的形状。

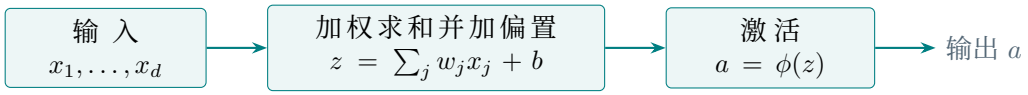


图 1.3: 单个神经元的计算。偏置在激活之前加入。

读图：输入先乘各自权重并相加，得到激活前的数值 z；激活函数 ϕ 将 z 转为输出 a。图中的“求和”并非把输入原样相加，而是进行加权求和。

1.3.1 先算一个单元

设 $\boldsymbol{x} = (2, -1)^\top$、$\boldsymbol{w} = (0.5, 1)^\top$、$b = 0.2$，则

$$z = \boldsymbol{w}^\top \boldsymbol{x} + b = 0.5 \times 2 + 1 \times (-1) + 0.2 = 0.2.$$

若用 ReLU，输出为 $a = 0.2$；若用 Sigmoid，输出约为 $a = 0.550$。相同的加权输入，经过不同的激活函数会产生不同的输出。

1.3.2 一层就是多个单元一起算

记第 l 层有 d_l 个单元，$\boldsymbol{a}^{(0)} = \boldsymbol{x}$。前向传播写为

$$\boldsymbol{z}^{(l)} = W^{(l)} \boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}, \qquad \boldsymbol{a}^{(l)} = \phi^{(l)}(\boldsymbol{z}^{(l)}). \tag{1.2}$$

对逐元素激活，每个单元只对自己的 z 做变换。Softmax 是向量变换，需把整组输出一起计算。

对象	形状	含义
$\boldsymbol{a}^{(l-1)}$	$d_{l-1} \times 1$	上一层输出
$W^{(l)}$	$d_l \times d_{l-1}$	每一行对应当前层一个单元的权重
$\boldsymbol{b}^{(l)}$	$d_l \times 1$	当前层偏置
$\boldsymbol{z}^{(l)}, \boldsymbol{a}^{(l)}$	$d_l \times 1$	当前层激活前的值与激活值

记住：先检查矩阵形状，再计算数值。隐藏层在计算新的表示；输出层把表示转换成任务所需的预测。

1.4 为什么需要非线性激活?

如果每层都只有仿射变换，堆叠后仍然只是仿射变换。例如

$$W_2(W_1\boldsymbol{x}+\boldsymbol{b}_1)+\boldsymbol{b}_2=(W_2W_1)\boldsymbol{x}+(W_2\boldsymbol{b}_1+\boldsymbol{b}_2).$$

因此，仅增加这样的层数无法获得一般的非线性表达能力。激活函数（Activation Function）让网络能表示更丰富的关系。



图 1.4: Sigmoid 平滑地压缩数值；ReLU 保留正半轴，把负半轴置零。

读图：左图过 $(0,0.5)$ 且单调递增，两端逐渐变平；右图在原点折转，负半轴为水平线，正半轴斜率为 1。曲线的斜率就是反向传播会用到的局部导数。

$$\sigma(z)=\frac{1}{1+e^{-z}},\qquad \sigma'(z)=\sigma(z)(1-\sigma(z)), \tag{1.3}$$

$$\mathrm{ReLU}(z)=\max(0,z),\qquad \mathrm{ReLU}'(z)=\begin{cases}0, & z<0,\\ 1, & z>0.\end{cases} \tag{1.4}$$

Sigmoid：有限实数输入的输出严格位于 $(0,1)$，适合表示单个二元事件的概率估计。其导数不超过 $1/4$；当 $|z|$ 很大时导数趋近于 0，但在有限实数处不等于 0。多层连乘小导数可能使梯度变小，因而 Sigmoid 通常不是深层隐藏层的默认选择。

修正线性单元（Rectified Linear Unit, ReLU）：计算简单，正半轴的导数为 1，是隐藏层常见的起点。$z=0$ 处经典导数不存在，实际自动求导通常约定取 0。如果一个单元对训练输入长期都输出 0，其输入侧参数可能难以再更新，这称为“死亡 ReLU”。

> **容易混淆**
>
> ReLU 不能保证整个深层网络不会出现梯度消失。梯度还受到权重、网络深度、输入分布等因素影响。Sigmoid 输出的数值也不会仅因位于 $(0,1)$ 就自动成为校准良好的概率。

1.5 Softmax：把一组得分变为分布

当一个样本必须属于 K 个互斥类别中的某一个时，网络通常先输出 K 个实数得分，称为 logits。Softmax 将这组得分转换为和为 1 的概率分布估计：

$$p_k = \mathrm{softmax}(\boldsymbol{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \dots, K. \tag{1.5}$$

对有限 logits，所有 $p_k > 0$ 且 $\sum_k p_k = 1$。分母遍历同一个样本的全部类别，而不是遍历不同样本。

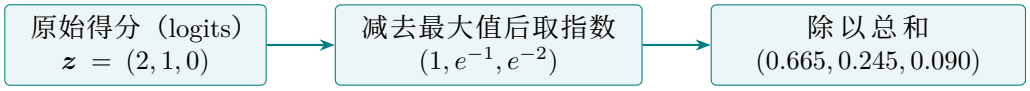


图 1.5: 三分类的数值例子。显示值经过四舍五入。

读图：得分最大的第一类获得最高概率，但其概率约为 0.665，并不接近 1。第二、三类仍保有概率质量。若取概率最大的类别作为最终标签，这一步才是离散的分类决策。

1.5.1 数值稳定性来自平移不变性

若给每个得分同时减去常数 c，分子分母会约去相同的因子 e^{-c}，所以结果不变。通常取 $c = \max_j z_j$：

$$p_k = \frac{e^{z_k - c}}{\sum_j e^{z_j - c}}. \tag{1.6}$$

这样所有指数的输入都不大于 0，避免直接计算过大的指数。图中的 $(2, 1, 0)$ 与 $(1002, 1001, 1000)$ 应当得到相同的分布。极小分量仍可能在有限精度下下溢；需要对数概率时，应直接使用稳定的 log-softmax 计算。

> **Softmax 会不会总选出一个接近 1 的概率？**
> 不会。若所有得分相等，每类概率都是 $1/K$；差距越大，分布才可能越集中。Softmax 保持得分的大小顺序，**它不是取最大值运算，也不是概率校准的保证**。例如把 $\boldsymbol{z}$ 乘以一个较大的正数，会改变概率集中程度，却不改变最高分的类别。

记住：Sigmoid 逐个处理二元事件；Softmax 联合处理互斥类别。判断任务能否同时有多个真标签，再选择输出。

1.6 把输出层和损失函数配对

输出层的形式由预测目标决定，不能只根据“分类”两个字选择。例如，一张图片的主类别若必须从猫、狗、鸟中选一个，是互斥多分类；若同时判断“有动物”“在室外”“有车辆”，则是多标签分类（Multilabel Classification）。

任务	输出维度	预测时常见变换	常见损失
无约束实值回归	目标数	恒等映射（线性输出）	均方误差
二分类	1	Sigmoid	二元交叉熵
互斥多分类	K	Softmax	类别交叉熵
多标签分类	标签数 K	每个分量分别用 Sigmoid	逐标签二元交叉熵

读表：维度是每个样本输出的数值个数。多标签任务的概率不必加起来等于 1；无约束实值回归才通常采用线性输出。正值目标、计数或其他带约束的目标，需要结合其统计模型选择输出和损失。二分类也能采用两个 logits 的 Softmax，这里列出的是更紧凑的一输出形式。

1.6.1 二元交叉熵（Binary Cross-Entropy, BCE）

对于 $y \in \{0, 1\}$，$p = \sigma(s)$ 是正类概率估计：

$$\ell_{\mathrm{BCE}}(s, y) = -y \log p - (1 - y) \log(1 - p). \tag{1.7}$$

这是**一个样本**的损失；数据集平均需另除以样本数 n，不会因为有两个类别而除以 2。本章的 log 均指自然对数。例如真实标签 $y = 1$ 时，$p = 0.9$ 的损失约为 0.105，$p = 0.1$ 的损失约为 2.303。

1.6.2 类别交叉熵（Categorical Cross-Entropy）

若真实类别索引为 y，则

$$\ell_{\mathrm{CE}}(\boldsymbol{z}, y) = -\log p_y = -\sum_{k=1}^{K} t_k \log p_k, \tag{1.8}$$

其中 $\boldsymbol{t}$ 是真实标签的独热向量。k 遍历类别，i 才用来遍历样本。例如图 1.5 若真实类别为第一类，损失约为 $-\log 0.665 = 0.408$。

> **公式里的概率与程序接口里的输入**
>
> 写概率形式便于理解；实际训练常把原始 logits 直接交给合并激活与交叉熵的稳定实现。例如 `BCEWithLogitsLoss` 接收 Sigmoid 前的得分，`CrossEntropyLoss` 接收 Softmax 前的得分。使用这类接口时，再手动做一次激活会改变计算。不同接口要求不同，应检查文档。

1.7 反向传播：把链式法则组织起来

反向传播（Backpropagation）高效地计算损失对参数的梯度。它沿前向计算的依赖关系反向工作，把已经算出的上游梯度与当前运算的局部导数组合，避免对共享的中间计算反复求导。

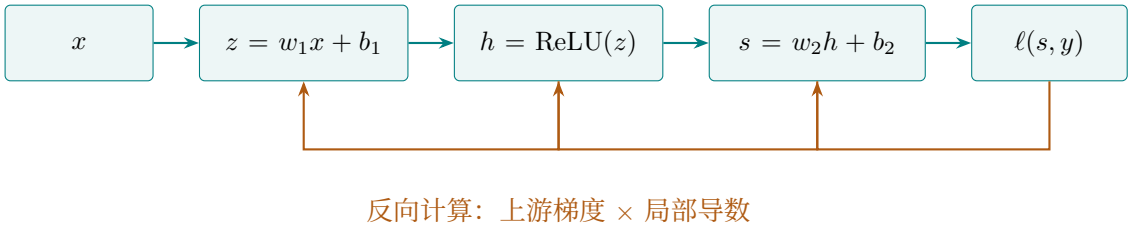


图 1.6: 一个标量小网络的计算图。青色向前计算数值，橙色反向传递导数。

读图：输入 x 先经过第一层和 ReLU 得到 h，再经过第二层得到得分 s，最后计算损失。反向时从 $\partial\ell/\partial s$ 开始，经第二层传到 h，再经 ReLU 传到 z。参数梯度从对应运算处分支得到。

对这一标量例子，链式法则给出

$$\frac{\partial \ell}{\partial w_1} = \frac{\partial \ell}{\partial s}\frac{\partial s}{\partial h}\frac{\partial h}{\partial z}\frac{\partial z}{\partial w_1}.$$

如果一个变量影响损失的路径不止一条，应将各路径的贡献相加。反向传播本身**不负责更新参数**；更新要由梯度下降、Adam 等优化器执行。

1.7.1 推广到全连接层

记 $\boldsymbol{\delta}^{(l)} = \partial\ell/\partial \boldsymbol{z}^{(l)}$，即损失对第 l 层激活前数值的梯度。对逐元素隐藏层激活，有

$$\boldsymbol{\delta}^{(l)} = \left((W^{(l+1)})^{\top}\boldsymbol{\delta}^{(l+1)}\right) \odot \phi'\left(\boldsymbol{z}^{(l)}\right), \tag{1.9}$$

$$\frac{\partial \ell}{\partial W^{(l)}} = \boldsymbol{\delta}^{(l)}(\boldsymbol{a}^{(l-1)})^{\top}, \qquad \frac{\partial \ell}{\partial \boldsymbol{b}^{(l)}} = \boldsymbol{\delta}^{(l)}. \tag{1.10}$$

其中 $\odot$ 表示逐元素乘法。外积得到的权重梯度形状为 $d_l \times d_{l-1}$，与 $W^{(l)}$ 相同。这些式子针对单个样本、未加正则化的损失；小批量平均损失的梯度也需取平均。

> **两个常见输出梯度**
>
> Sigmoid 配二元交叉熵时，$\partial\ell/\partial s = p - y$。Softmax 配类别交叉熵时，$\partial\ell/\partial \boldsymbol{z} = \boldsymbol{p} - \boldsymbol{t}$。这是“激活与相应损失组合”的结果，不能换一个损失后仍照抄。

1.8 手算一次完整的训练更新

现在用图 1.6 的网络做一次二分类训练。只用一个样本 $x = 2, y = 1$，初始参数为

$$w_1 = 0.5, \quad b_1 = 0, \quad w_2 = 1, \quad b_2 = 0.$$

隐藏层用 ReLU，输出得分经 Sigmoid 得到概率，损失为二元交叉熵。

1.8.1 第一步：前向计算

$$\begin{aligned} z &= w_1 x + b_1 = 1, & h &= \mathrm{ReLU}(1) = 1, \\ s &= w_2 h + b_2 = 1, & p &= \sigma(1) \approx 0.731059, \\ \ell &= -\log p \approx 0.313262. \end{aligned}$$

真实标签为 1，预测的正类概率还不够高，因此损失大于 0。

1.8.2 第二步：反向计算梯度

因为 $\partial \ell / \partial s = p - y \approx -0.268941$ 且 $z > 0$，所以 ReLU 的局部导数为 1：

$$\begin{aligned} \frac{\partial \ell}{\partial w_2} &= (p - y) h \approx -0.268941, & \frac{\partial \ell}{\partial b_2} &= p - y \approx -0.268941, \\ \frac{\partial \ell}{\partial w_1} &= (p - y) w_2 \times 1 \times x \approx -0.537883, & \frac{\partial \ell}{\partial b_1} &= (p - y) w_2 \times 1 \approx -0.268941. \end{aligned}$$

这四个梯度全部用**更新前的同一组参数**计算。例如计算第一层梯度时，使用的是旧的 $w_2 = 1$。

1.8.3 第三步：统一更新，再重新计算

取学习率 $\eta = 0.1$，每个参数减去学习率乘对应梯度：

$$(w_1', b_1', w_2', b_2') \approx (0.553788, 0.026894, 1.026894, 0.026894).$$

更新后重新前向计算，得到 $p' \approx 0.767$、$\ell' \approx 0.265$。这一步提高了正类概率，也降低了该训练样本的损失。这是本例的数值结果，并不是所有网络、所有学习率下每步都会下降的保证。

> **不要把反向传播拆成“边求梯度边更新”**
> 若先更新 w_2，再用新 w_2 求 w_1 的梯度，就不再是对同一个参数点计算整个梯度。通常应完成当前损失的反向计算，再让优化器统一更新。

1.9 梯度下降与学习率

梯度 $\nabla J(\theta)$ 指向目标函数局部增长最快的方向（采用欧氏距离衡量单位步长）。梯度下降（Gradient Descent）沿负梯度方向尝试降低目标：

$$\boxed{\theta_{t+1} = \theta_t - \eta_t \nabla J(\theta_t)}, \qquad \eta_t > 0. \tag{1.11}$$

t 为更新次数，η_t 为学习率（Learning Rate），控制梯度对参数更新的影响。实际步长为 $\eta_t \|\nabla J(\theta_t)\|$，不只由学习率决定。反向传播计算右侧的梯度，优化器执行这条或其他更新规则。

1.9.1 用一个二次函数看清学习率

令 $J(\theta) = \theta^2/2$，则 $J'(\theta) = \theta$，固定学习率下有 $\theta_{t+1} = (1-\eta)\theta_t$。取 $\theta_0 = 2$，得到下面三组**实际递推数值**。

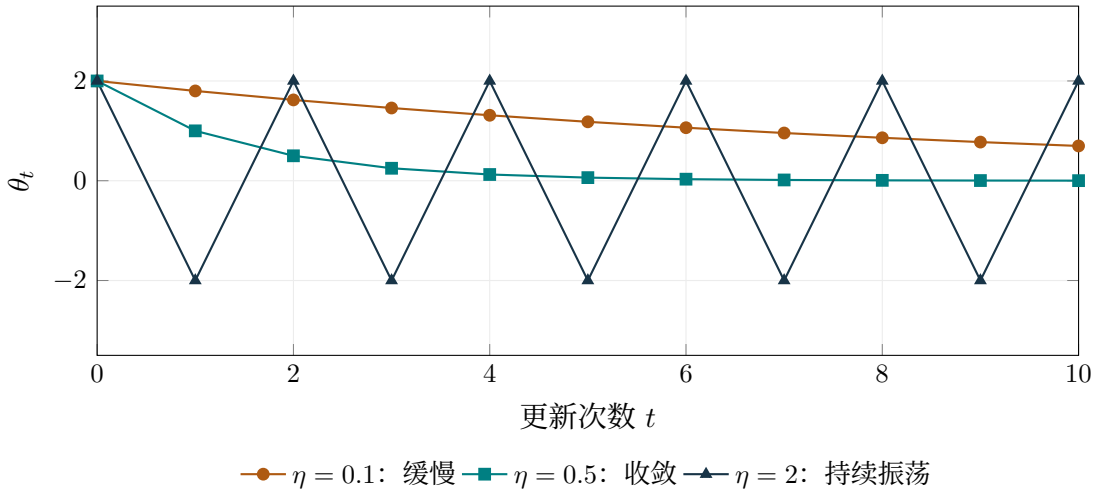


图 1.7: 纵轴是参数值，横轴是更新次数；最优参数为 $\theta = 0$。

读图： 橙线每次只缩小一点；青线较快接近 0；深色线不断在 2 与 -2 之间跳动，损失并未减小。这个例子中 $0 < \eta < 2$ 才收敛，$\eta > 2$ 则参数绝对值不断增大。

一般可微目标在足够小的步长下，负梯度方向具有局部下降意义。但神经网络目标通常非凸；不能据此保证到达全局最优。即使目标凸，也需要最小值存在、步长等条件，才能讨论具体算法的收敛。梯度为 0 只说明处于驻点，可能对应极小值、极大值或鞍点，损失也不必为 0。

从全批量到小批量

全批量梯度下降每步使用全部训练样本。小批量随机梯度下降（Minibatch SGD）用一小批样本平均梯度近似整体梯度，计算更便宜，但训练损失可能上下波动。调学习率和停止时刻应参考训练与验证表现，不用测试集反复试验。

1.10 一页速查与自测

问题	快速定位
网络怎样计算?	仿射变换与激活逐层组合；见1.3节。
为什么需要激活?	仿射变换堆叠仍是仿射变换；见1.4节。
Sigmoid 输出什么?	单个二元事件的概率估计，$\sigma(z) = 1/(1 + e^{-z})$。
Softmax 输出什么?	互斥类别的一组概率，分母遍历该样本的全部类别。
多标签怎么做?	常见起点是每个标签一个 Sigmoid，配逐标签 BCE。
反向传播做什么?	由损失计算梯度；每个梯度形状与对应参数一致。
优化器做什么?	按更新规则调整参数；普通梯度下降减去 $\eta\nabla J$。
训练损失越低越好?	还要检查验证表现；测试集用于最终评估。

1.10.1 三个自测问题

1. 一个 $5 \to 4 \to 2$ 的全连接网络，每个参数层都有偏置，共有多少参数?
2. 三个类别的 logits 都是 0，Softmax 输出是什么?
3. 若 $J(\theta) = \theta^2/2$，$\theta = 2$，$\eta = 0.1$，下一步参数是多少?

参考答案：（1）$(5 \times 4 + 4) + (4 \times 2 + 2) = 34$。（2）$(1/3, 1/3, 1/3)$。（3）$2 - 0.1 \times 2 = 1.8$。

1.10.2 进一步查阅

以下资料用于核对本章的计算机制与接口约定；本章图表和数值例子独立编写。

- 《动手学深度学习》：前向传播、反向传播与计算图。
- PyTorch 文档：BCEWithLogitsLoss 与 CrossEntropyLoss，核对 logits 输入及损失定义。